Disclaimer: These slides can include material from different sources. I'll be happy to explicitly acknowledge a source if required. Contact me for requests.

# Machine Learning in a Nutshell

## 15-288 Spring '26

Lecture 16:
Linear Models: Definitions, Properties, Training
Introduction to Gradient methods

Teacher:
Gianni A. Di Caro

جامعة كارنيجي ميلون في قطر
Carnegie Mellon University Qatar

Note:
The contents of slides 3 to 13 have already been covered (more or less) in previous lectures; here we review the concepts and provide additional info, especially regarding (non-linear) feature transformation 😎

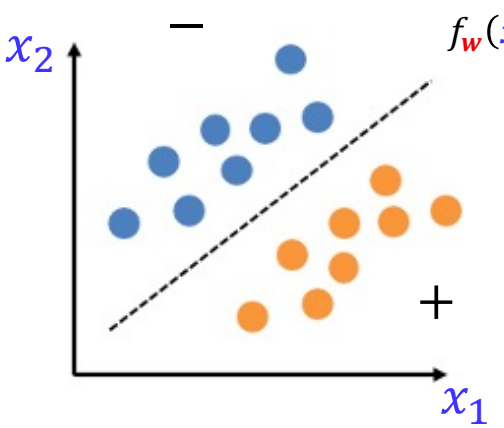
# Linear models: linear combination of input features

- We *assume* (i.e., we make the *hypothesis*) that the target output $f_{\boldsymbol{w}}(\boldsymbol{x})$ is *well described* by a weighted **linear combination** of the $d$ values of the **input features**

$$f_{\boldsymbol{w}}(\boldsymbol{x}) = w_1 x_1 + w_2 x_2 + \cdots + w_d\, x_d + w_0$$

**Classification**

Target output is a **Linear decision boundary**



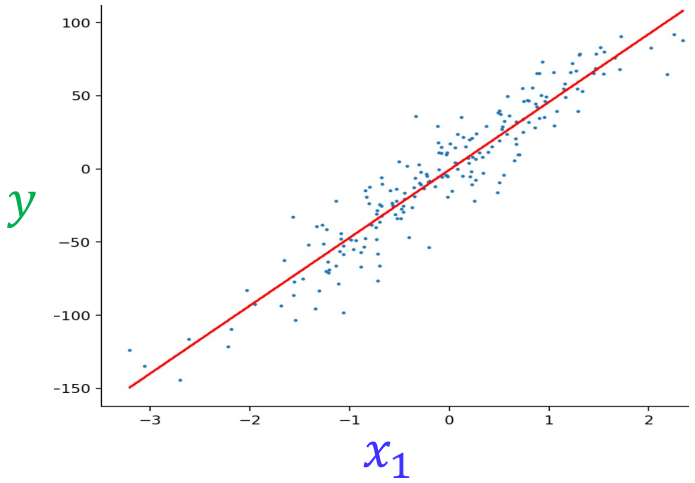

$f_{\boldsymbol{w}}(\boldsymbol{x}) = w_1 x_1 + w_2 x_2 + w_0$

Class label:

$y = \mathrm{sgn}(f_{\boldsymbol{w}}(\boldsymbol{x}))$

**Regression**

Target output is a **Linear curve fitting the data**




$f_{\boldsymbol{w}}(\boldsymbol{x}) = w_1 x_1 + w_0$

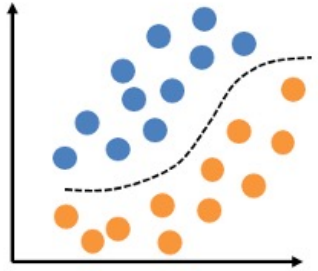
Value label:

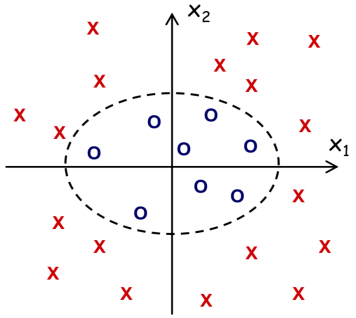
$y = f_{\boldsymbol{w}}(\boldsymbol{x})$

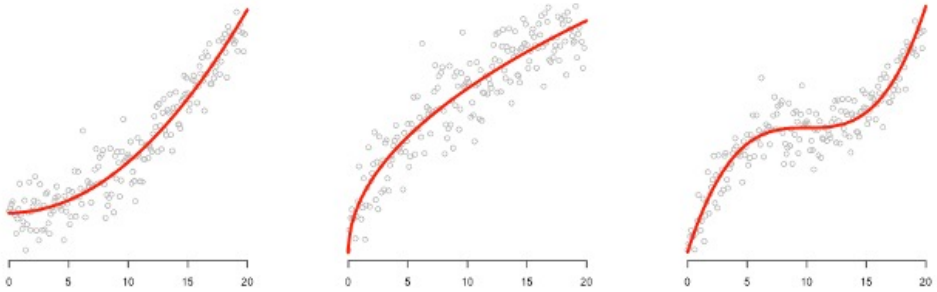
✓ With the **original / raw** features $x_1, x_2, \ldots, x_d$ the linear hypothesis works well IFF the data are linearly separable (for classification) or follow a linear pattern (for regression) in the feature space

# Linear models with transformations of input features

**Non-linearities in the original input feature space!**

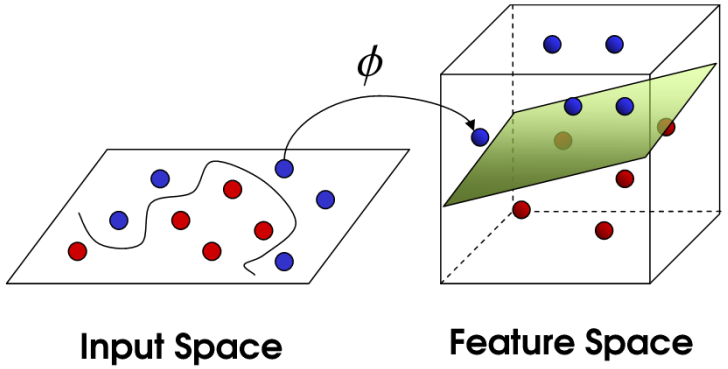


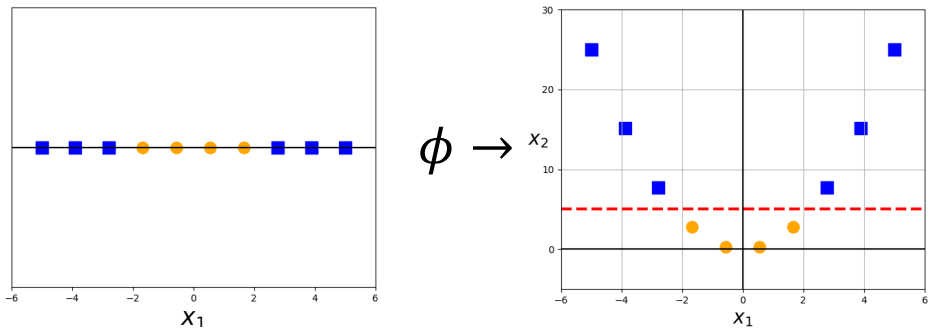


- We *assume* (i.e., we make the *hypothesis*) that the target output $f_{\boldsymbol{w}}(\boldsymbol{x})$ is *well described* by a weighted **linear combination** of some $m$-dimensional **function** $\phi_i$ of the **input features**

$$f_{\boldsymbol{w}}(\boldsymbol{x}) = w_1\phi_1(\boldsymbol{x}) + w_2\phi_2(\boldsymbol{x}) + \cdots + w_m\,\phi_m(\boldsymbol{x}) + w_0$$



# Linear models: Feature transformation

<u>Goal:</u> Find the *best* **linear function in the feature space**

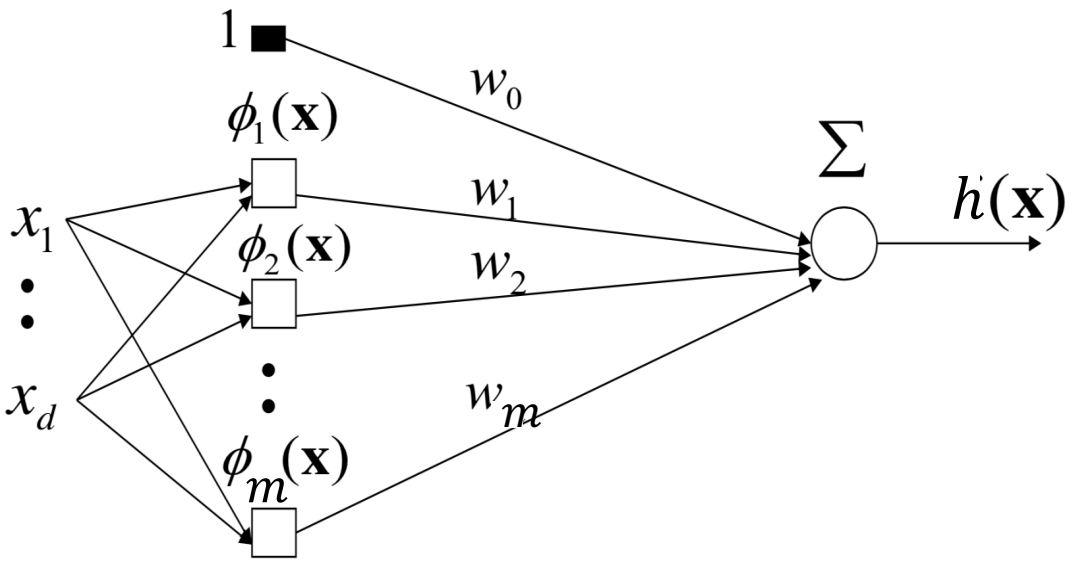
$$f_{\boldsymbol{w}}(\boldsymbol{x}) = w_1\phi_1(\boldsymbol{x}) + w_2\phi_2(\boldsymbol{x}) + \cdots + w_m\,\phi_m(\boldsymbol{x}) + w_0$$

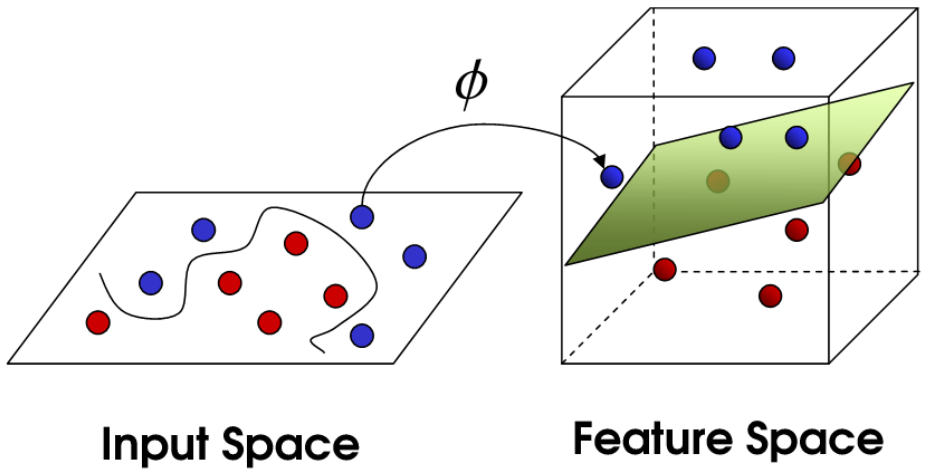
$\boldsymbol{x} \in \mathbb{R}^d$

$\boldsymbol{w} \in \mathbb{R}^m$

$\phi_i : \mathbb{R}^d \to \mathbb{R}, \qquad i = 1, \cdots, m$

Feature functions



# Linear models: Feature transformation

<u>Goal:</u> Find the *best* **linear function in the feature space**

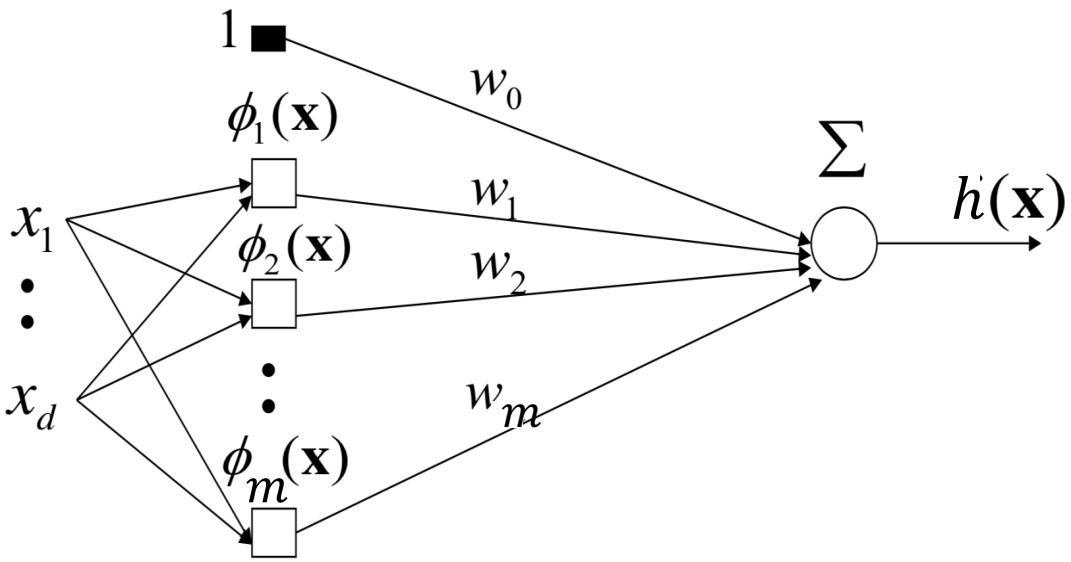
$$f_{\boldsymbol{w}}(\boldsymbol{x}) = w_1\phi_1(\boldsymbol{x}) + w_2\phi_2(\boldsymbol{x}) + \cdots + w_m\,\phi_m(\boldsymbol{x}) + w_0$$

$\boldsymbol{x} \in \mathbb{R}^d$

$\boldsymbol{w} \in \mathbb{R}^m$

$\phi_i : \mathbb{R}^d \to \mathbb{R}, \qquad i = 1, \cdots, m$

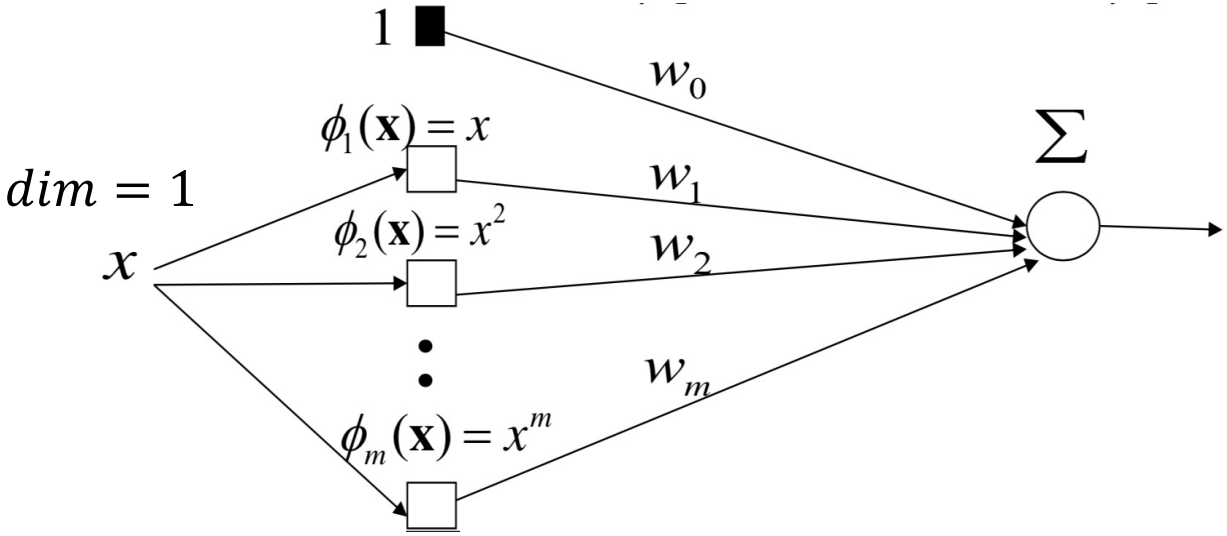
Feature functions




1. Can the $\phi_i$ be any type of function, or do they need to be *restricted* to some specific classes of functions?
2. What is the dimensionality of the feature space?

# Linear models: Feature transformation

- Original features space: $x \in \mathbb{R}$ (1D)
- E.g., $m$ **polynomial functions** with 1D input, $\boldsymbol{x} = (x)$
  - $\phi_1(\boldsymbol{x}) = x$, $\phi_2(\boldsymbol{x}) = x^2$, $\phi_3(\boldsymbol{x}) = x^3$, $\cdots$




$dim = m \gg 1$

# Linear models: Feature transformation

$$f_{\boldsymbol{w}}(\boldsymbol{x}) = w_1\phi_1(\boldsymbol{x}) + w_2\phi_2(\boldsymbol{x}) + \cdots + w_m\,\phi_m(\boldsymbol{x}) + w_0$$

$\boldsymbol{x} \in \mathbb{R}^d$

$\boldsymbol{w} \in \mathbb{R}^m$

$\phi_i : \mathbb{R}^d \to \mathbb{R}, \qquad i = 1, \cdots, m$

- $\{\phi_i\,, i = 1, \ldots, m\}$ = basis functions of the **new** feature space, $\boldsymbol{z} = \Phi(\boldsymbol{x})$

$$\left.\begin{aligned} z_1 &= \phi_1(\boldsymbol{x}) \\ z_2 &= \phi_2(\boldsymbol{x}) \\ &\cdots \\ z_m &= \phi_m(\boldsymbol{x}) \end{aligned}\right\} \quad \begin{aligned} &\Phi : \boldsymbol{x} \in \mathbb{R}^d \to \boldsymbol{z} \in \mathbb{R}^m \\ &\Phi = (\phi_1, \phi_2, \ldots, \phi_m) \end{aligned}$$

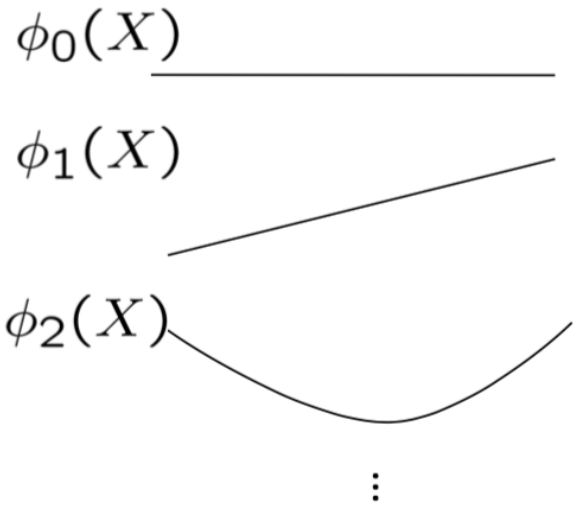
$$\boxed{f_w(\boldsymbol{z}) = w_1 z_1 + w_2 z_2 + \cdots + w_m z_m + w_0}$$

✓ <u>Linear classifier</u> in the $m$-dim feature space $\boldsymbol{z} = \Phi(\boldsymbol{x})$

# Linear models: Feature transformation

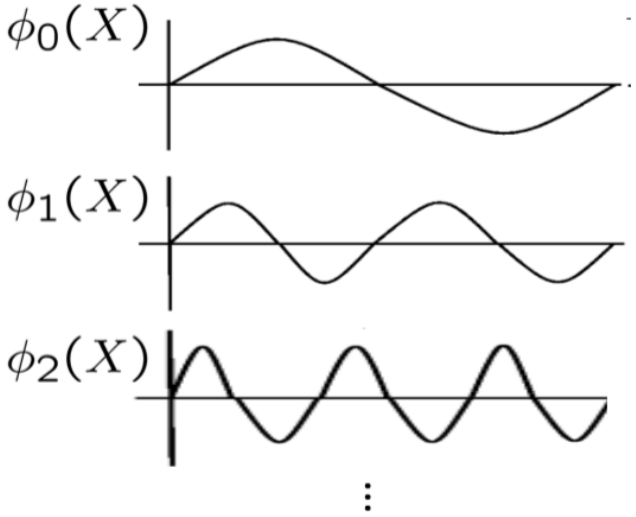
- $\{\phi_i\ , i = 1, \dots, m\}$ = **basis functions** of the **new** feature space, $\boldsymbol{z} = \Phi(\boldsymbol{x})$

Polynomial Basis




$\phi_k(\mathrm{x}) = \mathrm{c} + \mathrm{x}^k$

Fourier Basis

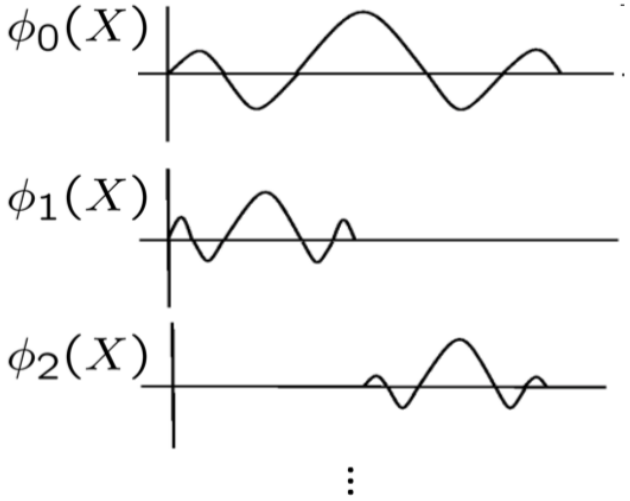



Good representation for periodic functions

$\phi_k(\mathrm{x}) = \sin(k\mathrm{x})$

$\psi_k(\mathrm{x}) = \cos(k\mathrm{x})$

Wavelet Basis




Good representation for local functions

# (Extras) Notes on Basis functions

A set of functions $\{\phi_k\ , k = 1, \dots, m\}$ is called a **basis** for a function space $\mathcal{F}$ if **any function** $f(x) \in \mathcal{F}$ can be expressed as a <u>linear combination</u> of these basis functions (note that $m$ can be equal to $\infty$)

$$f(x) = \sum_{k=1}^{m} c_k \phi_k(x)$$

where $c_k$ are coefficients that depend on $f(x)$ and the chosen basis.

- ✓ If we use basis functions, we can **approximate with arbitrary precision *any* function in** $\mathcal{F}$!

- ➢ However, the basis might easily need infinite functions $\phi_k(x)$ to be able to perform such a task, such that the sum goes over an infinite number of terms: $\sum_{k=1}^{\infty} c_k \phi_k(x)$

- ➢ In the language of the features, we might need an *infinite number of features* (to define, compute, and store), which clearly can't happen with a manual transformation 😏

# (Extras) Notes on Basis functions

- The **Taylor series** of a function $f(x)$ around $x = a$ can approximate the function to an arbitrary degree:

$$f(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(a)}{k!}(x-a)^k$$

In the expansion the basis functions are **monomials** $\phi_k = (x-a)^k$ and the coefficients are $c_k = \frac{f^{(k)}(a)}{k!}$

- ➢ The coefficients determine how much each basis function contributes to the linear combination.
- ❖ If the sum is performed up to $k = d < \infty$, the **truncated Taylor series** approximates the function up to degree $d$ (e.g., if $d = 2$, it represents a quadratic approximation)

- A **Fourier basis** is a set of sinusoidal functions (sines and cosines) that form an orthonormal basis for representing <u>periodic functions</u>. The key idea is that any periodic function (or more generally, <u>any function</u> satisfying certain conditions) can be expressed as a sum of these basis functions.

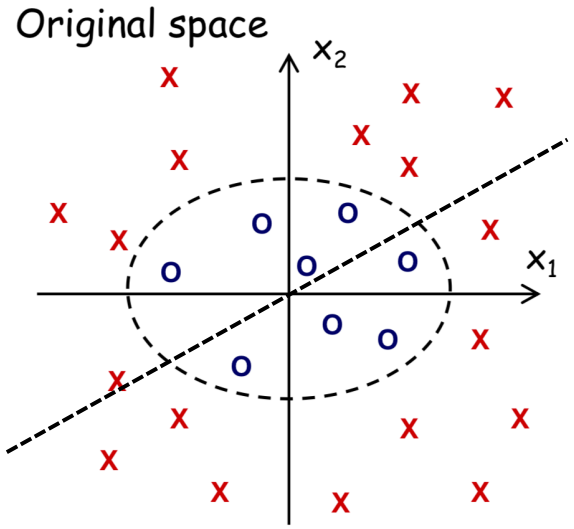
$$f(x) = a_0 + \sum_{k=1}^{\infty} (a_k \cos(kx) + b_k \sin(kx))$$

In the series the basis functions are $\phi_k = \cos(kx)$ , $\psi_k = \sin(kx)$ and the coefficients are $a_k, b_k$

# Linear models: Feature transformation

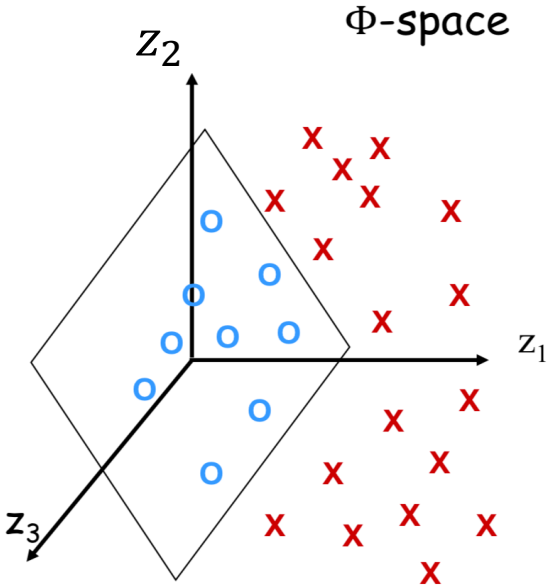
- $\{\phi_i\ , i = 1, \dots, m\}$ = basis functions of the **new** feature space, $\boldsymbol{z} = \Phi(\boldsymbol{x})$

  - ✓ IF the transformation is well chosen, in the **new feature space** $\Phi$ data examples are **linearly separable (for classification) or linearly generalizable (for regression)**




$\Phi$: $\mathbb{R}^2 \rightarrow \mathbb{R}^3$

$\Phi : (x_1, x_2) \rightarrow (z_1, z_2, z_3) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$

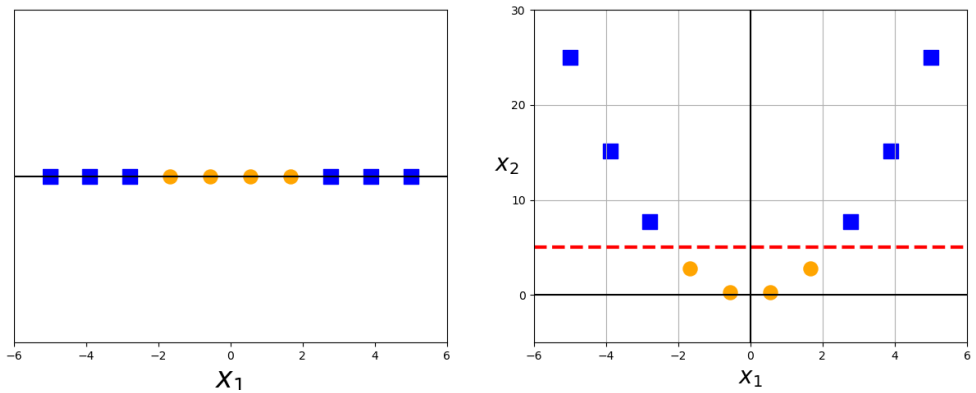



A (2D) linear classifier would perform very poorly

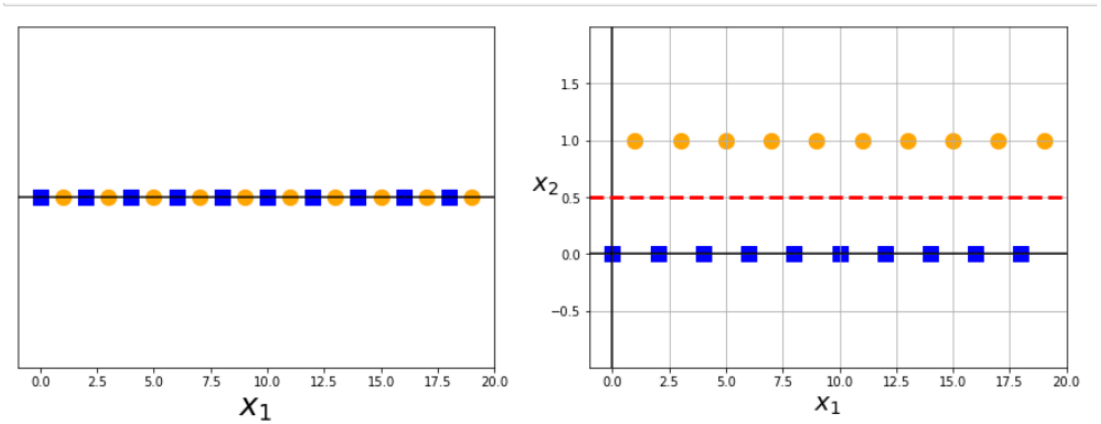
✓ In the 3D $\Phi$ space, a linear classifier (hyperplane) perfectly separates the data

# Linear models: Feature transformation

- $\{\phi_i\ , i = 1, \dots, m\}$ = basis functions of the **new** feature space, $\boldsymbol{z} = \Phi(\boldsymbol{x})$




$\phi(x) = (x, x^2)$



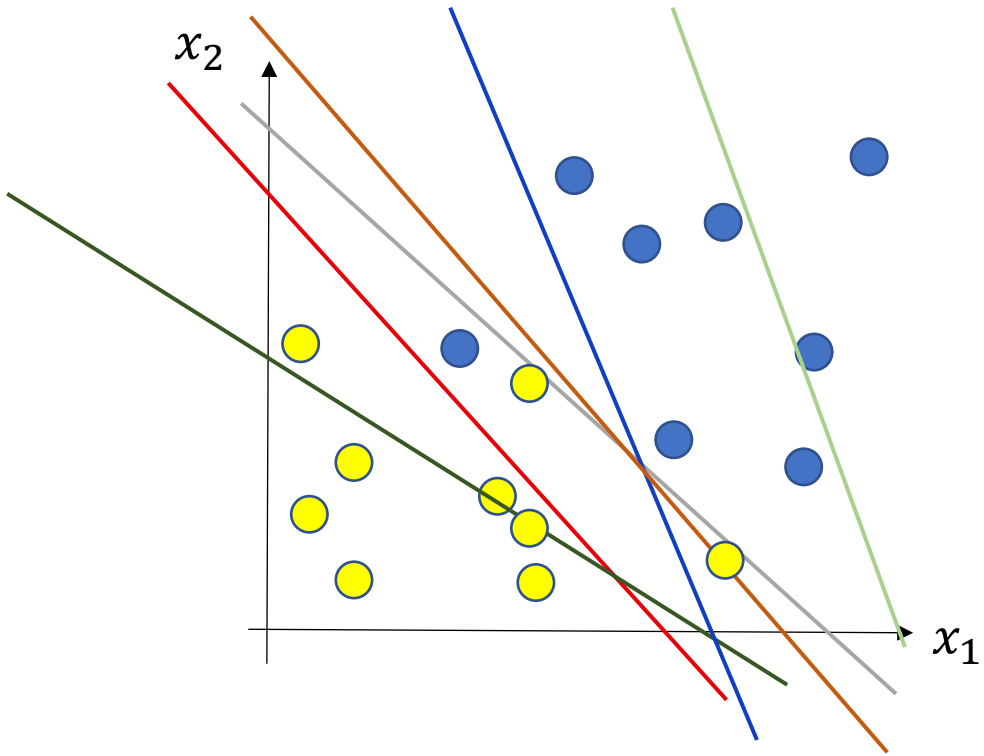

$\phi(x) = (x, x \bmod 2)$

Note:
The contents of slides 15 to 31 have already been covered in previous lectures using same slides (more or less). Here we review the concepts to move forward with the concepts of score and margin of a classifier 😎

# Linear models for classification

We have already explored linear models for **regression**, let's focus now on linear models for **classification**!

Find the *best* values for $\boldsymbol{w}$ (slope, intercept) using the training data



# Linear models: formulation with and without bias

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

Input features

$$f_{\boldsymbol{w}}(\boldsymbol{x}) = w_0 \cdot 1 + w_1 x_1 + w_2 x_2 + \cdots + w_d\, x_d$$

$$\boldsymbol{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix}$$

Learned weights
Coefficients of linear function

$$f_{\boldsymbol{w}}(\boldsymbol{x}) = \boldsymbol{w}^T \boldsymbol{x} + w_0$$

$$f_{\boldsymbol{w}}(\boldsymbol{x}) = \boldsymbol{w}^T \boldsymbol{x} + b \qquad w_0 \equiv b = \textbf{bias}$$

$$\boldsymbol{x}' = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

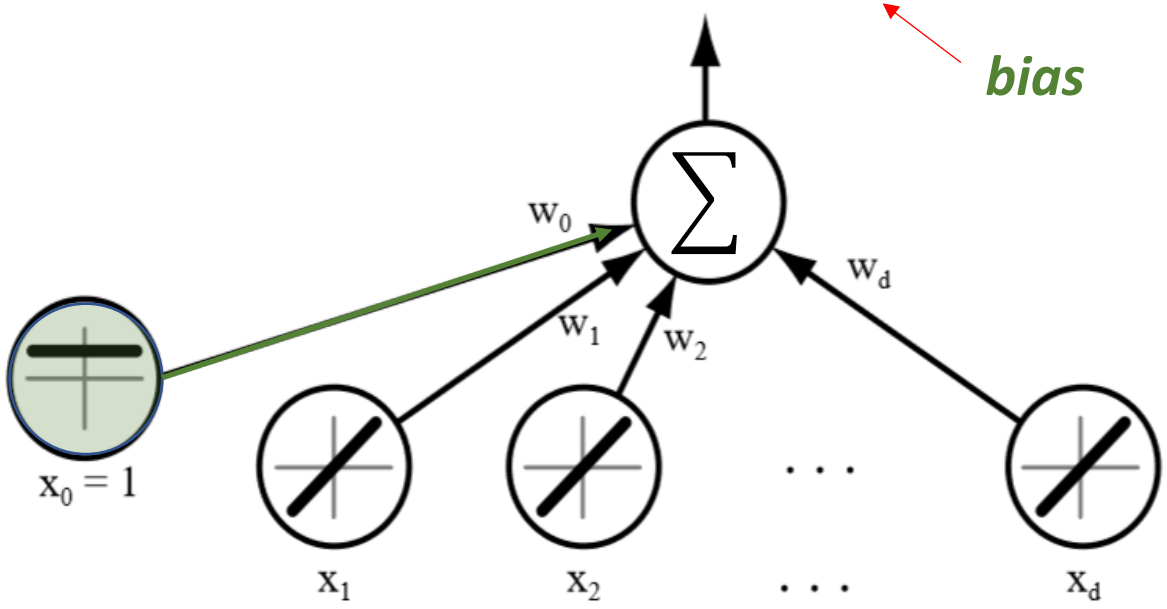
$$f_{\boldsymbol{w}}(\boldsymbol{x}) = \boldsymbol{w}^T \cdot \boldsymbol{x}'$$

Compact form, as **single dot product**

# Linear models: visualization of additive aggregator and bias

$$f_{\boldsymbol{w}}(\boldsymbol{x}) = w^T \cdot x + w_0 \cdot 1$$

***bias***




$$\left.\begin{aligned} f_{w_0} &= w_0 \\ f_{w_1}(x_1) &= w_1 x_1 \\ f_{w_2}(x_2) &= w_2 x_2 \\ &\dots \\ f_{w_d}(x_d) &= w_d x_d \end{aligned}\right\} \quad f_{\boldsymbol{w}}(\boldsymbol{x}) = \sum_{i=1}^{d} w_i x_i + w_0$$

Bias weight: Independent of the features/input
→ A fixed amount added to the sum

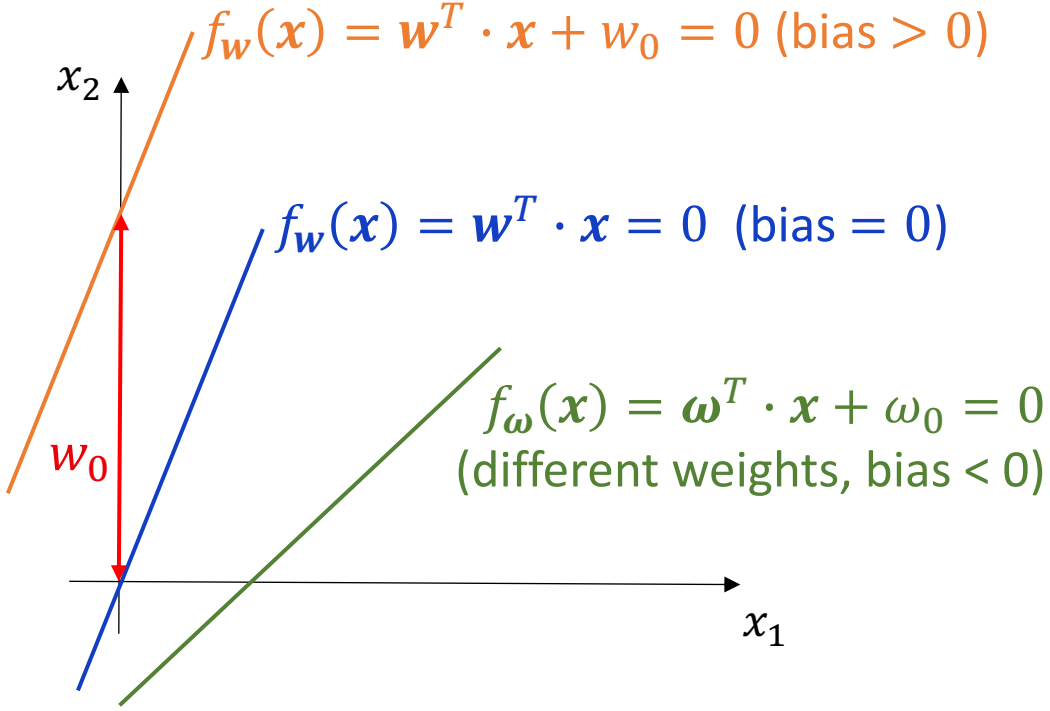
$d + 1$ linear functions,
one for each feature

# Linear models: geometry

$y = ax + b \quad \rightarrow \quad ax_1 - 1x_2 + b = 0$ (implicit form) $\quad w_1x_1 + w_2x_2 + w_0 = 0$

$f_{\boldsymbol{w}}(\boldsymbol{x}) = w_1x_1 + w_2x_2 + w_0 = 0$

$f_{\boldsymbol{w}}(\boldsymbol{x}) = \boldsymbol{w}^T\boldsymbol{x} + w_0 = 0$

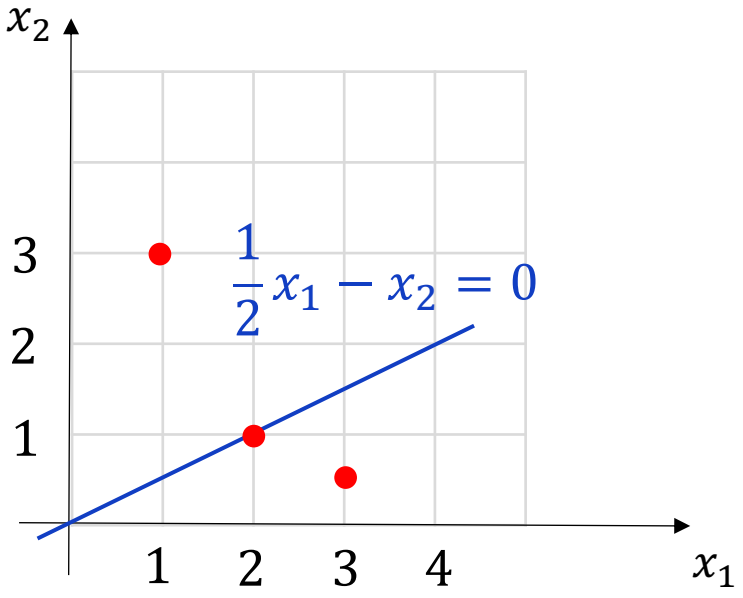


# Linear models for Classification: How to?

$f_{\boldsymbol{w}}(\boldsymbol{x}) = \boldsymbol{w}^T\boldsymbol{x} + b,$ $\quad f_{\boldsymbol{w}}(\boldsymbol{x}) \in \mathbb{R}$, in general can be *any* value between $]-\infty, +\infty[$

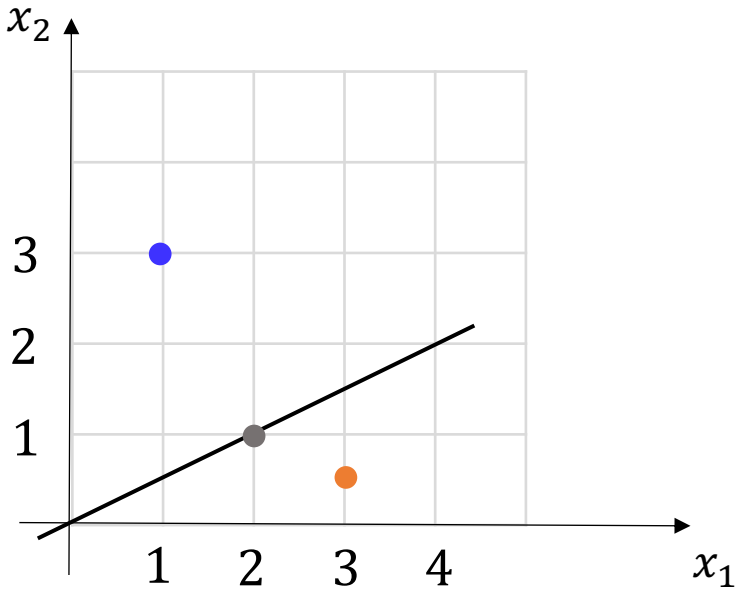
- For a 2-dimensional feature space: $f_{\boldsymbol{w}}(\boldsymbol{x}) = w_1 x_1 + w_2 x_2 + w_0$
  - E.g., $\boldsymbol{w} = (0, \frac{1}{2}, -1)$ $\quad f_{\boldsymbol{w}}(\boldsymbol{x}) = \frac{1}{2}x_1 - 1x_2 + 0$

- $f_{\boldsymbol{w}}(2,1) = 0$
- $f_{\boldsymbol{w}}(3,0.5) = 1 > 0$
- $f_{\boldsymbol{w}}(1,3) = -\frac{5}{2} < 0$
- $f_{\boldsymbol{w}}(100,1) = 99 \gg 0$

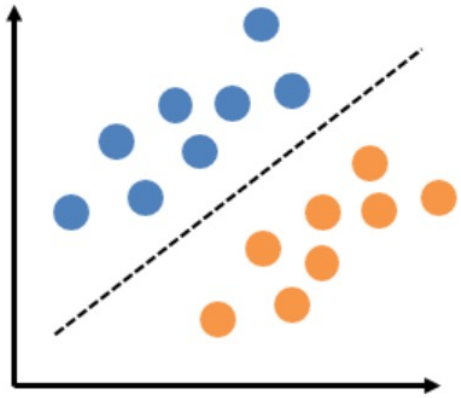


# Linear models for Classification (binary): How to?

- $f_{\boldsymbol{w}}(2,1) = 0$
- $f_{\boldsymbol{w}}(3,0.5) = 1 > 0$
- $f_{\boldsymbol{w}}(1,3) = -\frac{5}{2} < 0$
- $f_{\boldsymbol{w}}(100,1) = 99 \gg 0$




How do we use a linear model $f_{\boldsymbol{w}}$ for *classification*? (discrete set of labels!)



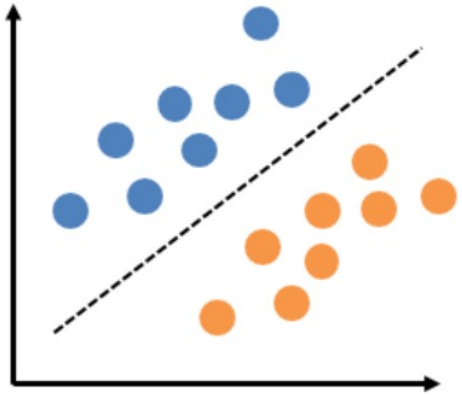
# Fundamental questions about linear classifiers

1. How do we use a linear classifier to **make predictions**?
   - Which **classes** correspond to the values $1, -\frac{5}{2}, 99, \ldots$ ?

2. How do we **train / learn** a linear classifier?
   - How do we find the ***best*** linear decision boundary?

# Linear models for Classification (binary)

1. How do we use it to **make predictions**?
   - Which **classes** correspond to the values $1, -\frac{5}{2}, 99, \ldots$ ?

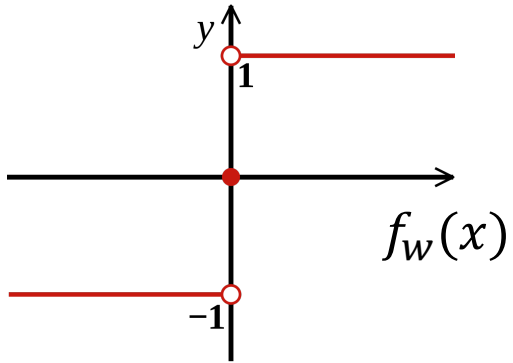


→ Need a *quantization / discretization* of the <u>continuous range</u> of values of $f_{\boldsymbol{w}}(\boldsymbol{x})$ in a set of <u>two values</u>

Note the procedures for the case of more than 2 classes are similar and can build on the binary case

➢ **Transformation** $\mathrm{sgn}(f_{\boldsymbol{w}}(\boldsymbol{x}))$ naturally outputs $\{-1, +1\}$

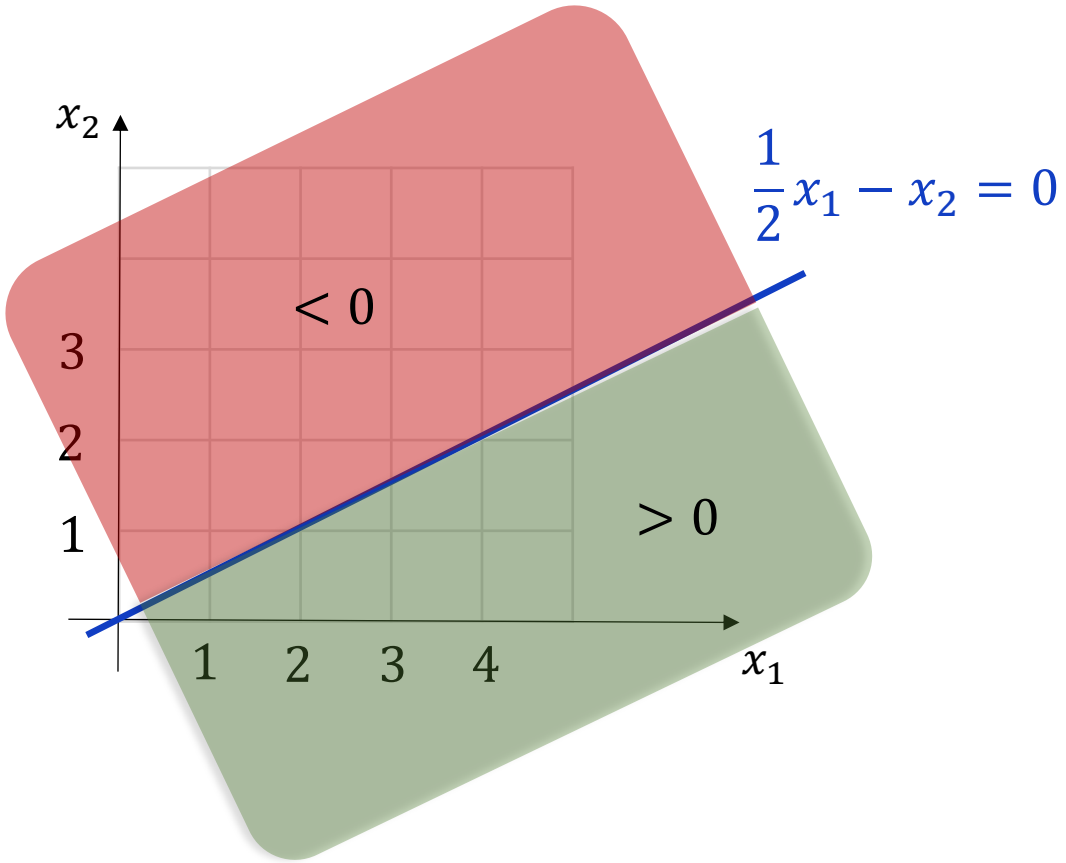
$$\mathrm{sgn}\big(f_{\boldsymbol{w}}(\boldsymbol{x})\big) : \boldsymbol{x} \rightarrow y \in \{-1, +1\}$$



# Linear models: basic geometry for binary case

$$f_{\boldsymbol{w}}(\boldsymbol{x}) = w_1 x_1 + w_2 x_2 + w_0 = 0$$

$$f_{\boldsymbol{w}}(\boldsymbol{x}) = \frac{1}{2} x_1 - x_2 = 0 \qquad \boldsymbol{w} = (0, \frac{1}{2}, -1)$$

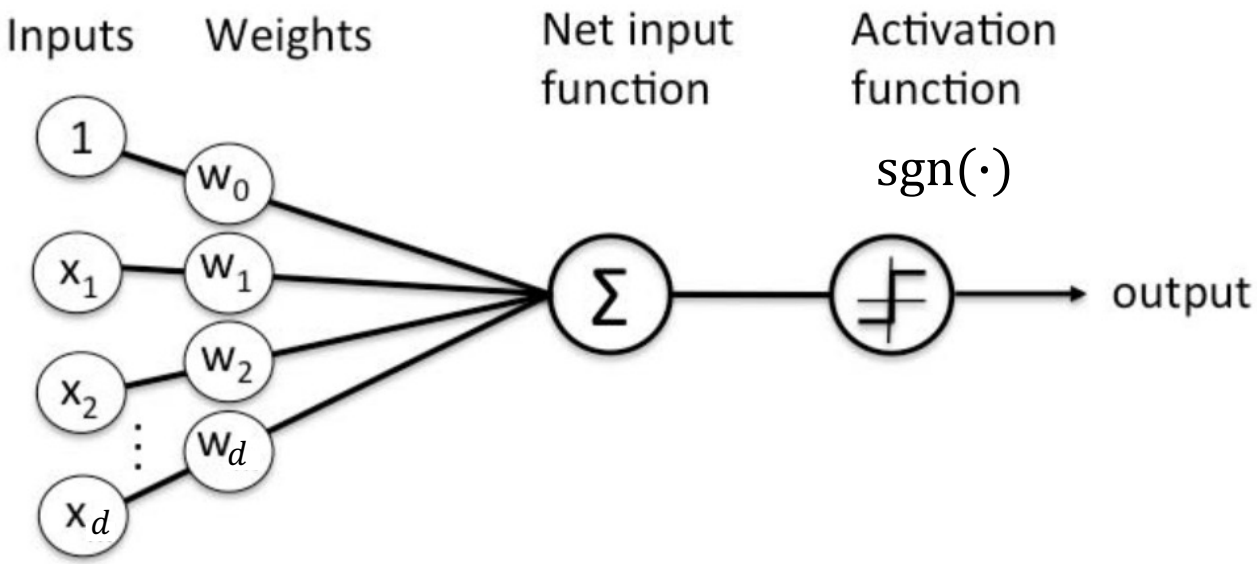



- ❑ $\boldsymbol{x} = (2, 1)$ is **on the line**
  - ✓ $f(\boldsymbol{x}) = 1 - 1 = 0$
- ❑ $\boldsymbol{x} = (3, 0.5)$ in **linear half-space** where $f_{\boldsymbol{w}}(\boldsymbol{x}) > 0$
  - ❖ $f_{\boldsymbol{w}}(\boldsymbol{x}) = \frac{3}{2} - \frac{1}{2} = 1 > 0$
- ❑ $\boldsymbol{x} = (1, 3)$ in **linear half-space** where $f_{\boldsymbol{w}}(\boldsymbol{x}) < 0$
  - ❖ $f_{\boldsymbol{w}}(\boldsymbol{x}) = \frac{1}{2} - 3 = -\frac{5}{2} < 0$

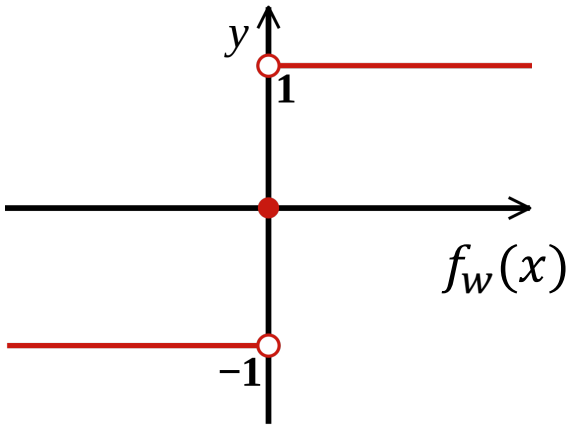
✓ The line of equation $f_{\boldsymbol{w}}(\boldsymbol{x})$ separates the region of the inputs $\boldsymbol{x}^+$, that determine a **positive value** of $f_{\boldsymbol{w}}(\boldsymbol{x})$, from the region of the inputs $\boldsymbol{x}^-$, that determine a **negative value** of $f_{\boldsymbol{w}}(\boldsymbol{x})$

# Linear models for Classification (binary)

$$f_{\boldsymbol{w}}(\boldsymbol{x}) = \boldsymbol{w}^T\boldsymbol{x} + b$$



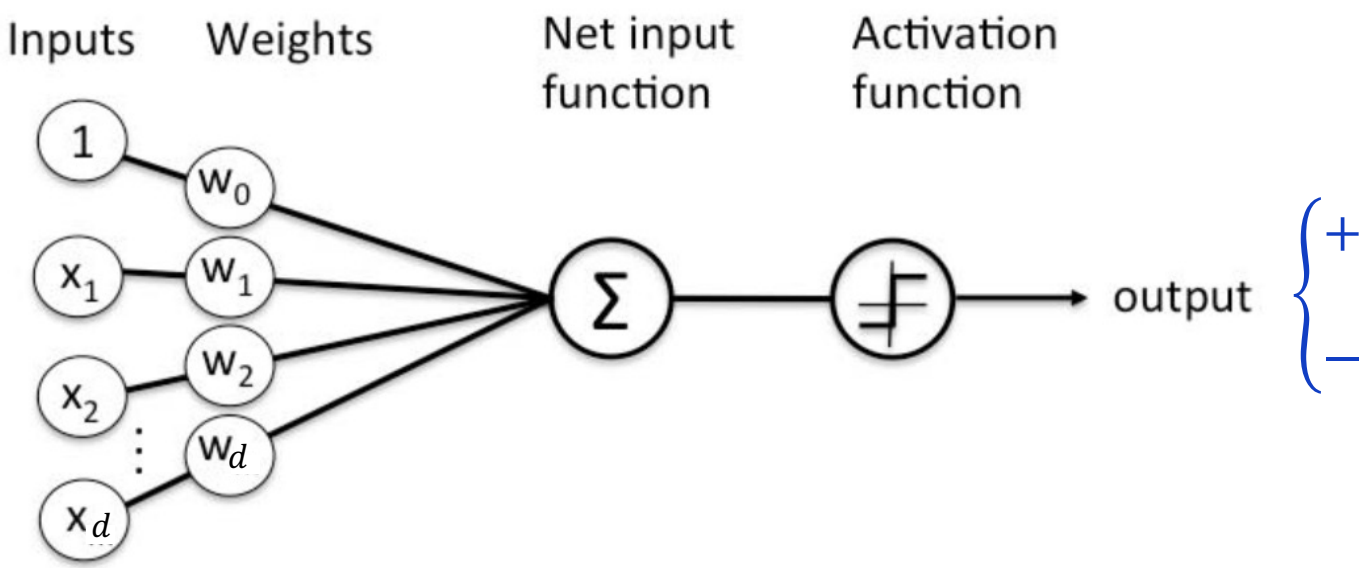

$$y = \begin{cases} + \\ - \end{cases} \qquad y = \mathrm{sgn}(\boldsymbol{w}^T\boldsymbol{x} + b) = \mathrm{sgn}\Bigg(\overbrace{\sum_{j=1}^{d} w_j x_j + b}^{f_{\boldsymbol{w}}(\boldsymbol{x})}\Bigg)$$




$$y = \begin{cases} 1 & if \quad \sum_{j=1}^{d} w_j x_j \geq threshold \\ -1 & if \quad \sum_{j=1}^{d} w_j x_j < threshold \end{cases}$$

threshold ≡ **bias** ≡ $-b$

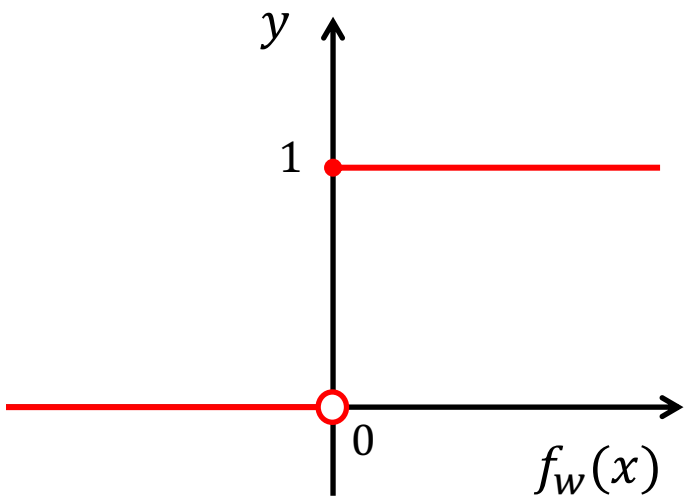
# Linear models for Classification (binary)




$\begin{cases} + \\ - \end{cases}$

Other transformations of $f_w(x)$ can be used, as long as the output $y$ is a two-value set

- **Class labels $y$**: $\{0,1\}$, $\{-1,1\}$, … $\{a,b\}$, $\{+,-\}$



# Linear classifier: Should I stay, or should I go?

1. How do we use a liner classifier to **make predictions**?

- Need to decide if I should stay at home or got to watch a movie, binary decision-making
  - $x_1$ = Is Al Pacino playing in the movie? $\{0,1\}$
  - $x_2$ = Is Scarlett Johansson playing in the movie? $\{0,1\}$
  - $x_3$ = Is a mafia or thriller movie? $\{0,1\}$
  - $x_4$ = How many friends are willing to come with me? $\{0, \cdots, n\}$
  - $x_5$ = Am I busy? $[0,1]$
  - $x_6$ = How far the cinema theater is? $[5, 60]$

- $w_1 = 0.8$
- $w_2 = 0.9$
- $w_3 = 0.75$
- $w_4 = 0.3$
- $w_5 = -0.8$
- $w_6 = -0.02$

Preferences / experience-based weights

*Threshold/bias* = $-0.9$

$\boldsymbol{x} = (0, 1, 0, 1, 0.6, 25)$

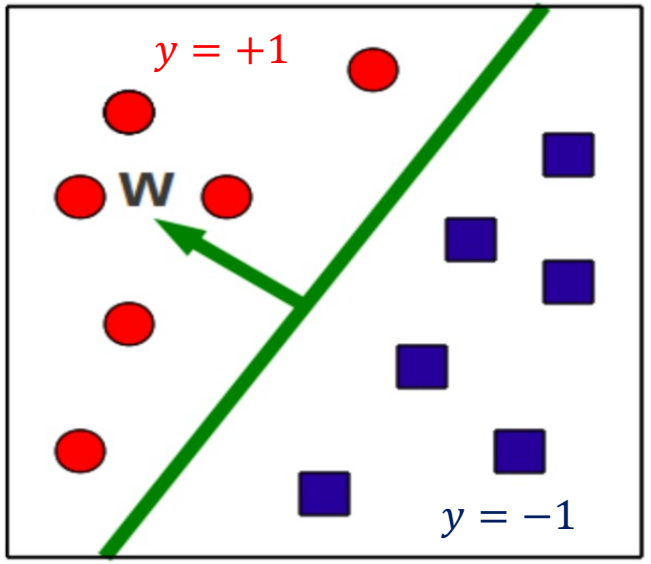
$\boldsymbol{w}^T\boldsymbol{x} = 0 + 0.9 + 0 + 0.7 - 0.18 - 0.5 = 0.92$

I will go!

# Linear models for classification: Geometry of $\boldsymbol{w}$

<u>Goal:</u> Find the *best* **linear decision boundary (hyperplane)**

→ Find the *best* values for $\boldsymbol{w}$ using the training data




$$f_{\boldsymbol{w}}(\boldsymbol{x}) = \boldsymbol{w}^T \cdot \boldsymbol{x}'$$

$$\boldsymbol{x}' = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

❖ Geometrically $\boldsymbol{w}$ is a vector **orthogonal** to the line/hyperplane identified by $f_{\boldsymbol{w}}(\boldsymbol{x})$

- The equation of the line is $f_{\boldsymbol{w}}(\boldsymbol{x}) = \boldsymbol{w}^T \cdot \boldsymbol{x}' = 0$, which is the **scalar product between two vectors**, and the scalar product is 0 when the two vectors are orthogonal
- i.e., the scalar product between $\boldsymbol{w}$ and any point vector $\boldsymbol{x}$ on the line is zero → $\boldsymbol{w}$ is ⊥ to the line/hyperplane

# Fundamental questions about linear classifiers

1. How do we use a linear classifier to **make predictions**?
   - Which **classes** correspond to the values $1, -\frac{5}{2}, 99, \ldots$ ?

2. How do we **train / learn** a linear classifier?
   - How do we find the ***best*** linear decision boundary?
   - The ***best***?

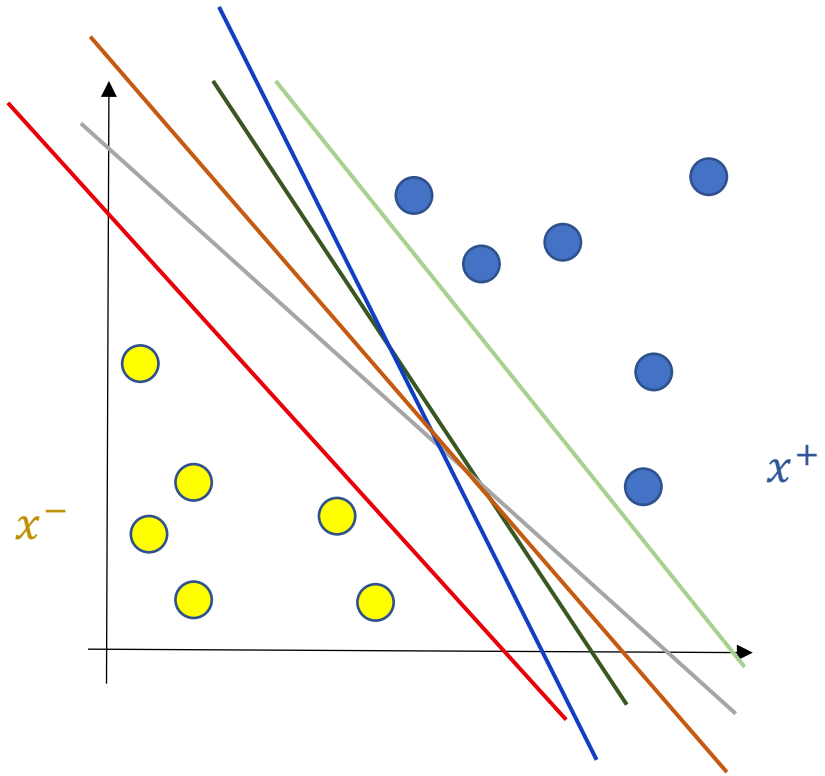
# Which linear classifier?

Which linear classifier is the *best* given the data?

- ✓ **Consistent:** it agrees with the data → <u>Minimizes the error on the training set</u>
  - *Fully consistent* if data is linearly separable, *partially consistent* if data isn't linearly separable
- ✓ ***Generalize* well:** it makes correct predictions on new, test data

Which linear separator is the best one? (right to left)

1. Green
2. Blue
3. Black
4. Orange
5. Gray
6. Red



# Recap: The Canonical Supervised ML Problem

1. Given a **dataset of training examples**, $D$, consisting of pairs of input features and target outputs $\{(\boldsymbol{x}^{(i)}, y^{(i)})\},\ i = 1, \dots, m$, and selected:
   - a **hypothesis function** $f_{\boldsymbol{w}}(\boldsymbol{x})$,
   - a **loss function** $\ell(f_{\boldsymbol{w}}(\boldsymbol{x}), y)$

   find (or approximate) **parameter values** $\boldsymbol{w}^*$ that **minimize the empirical error on** $D$:

$$\boldsymbol{w}^* = \min_{\boldsymbol{w}} \frac{1}{m} \sum_{i=1}^{m} \ell\left(f_{\boldsymbol{w}}\left(\boldsymbol{x}^{(i)}\right), y^{(i)}\right)$$

2. If **validation sets** of examples, $V$, are made available (<u>always possible in some way</u>), find the parameter values $w^*$ that are the best ***tradeoff*** between:
   - **minimizing the empirical error on the training set**
   - and at the same time **minimize the empirical error on the validation sets**

# The Canonical Supervised ML Problem for linear hypotheses

- Given a **dataset of training examples**, $D$, consisting of pairs of input features and target outputs $\{(\boldsymbol{x}^{(i)}, y^{(i)})\},\ i = 1, \dots, m$, and selected:
  - a **hypothesis function** $f_{\boldsymbol{w}}(\boldsymbol{x}) = \boldsymbol{w}^T \cdot \boldsymbol{x}'$ ,
  - a **loss function** $\ell(\boldsymbol{w}^T \cdot \boldsymbol{x}',\ y) = ?$

  find (or approximate) **parameter values** $\boldsymbol{w}^*$ that **minimize the empirical error on** $D$:

$$\boldsymbol{w}^* = \min_{\boldsymbol{w}} \frac{1}{m} \sum_{i=1}^{m} \ell\left(\boldsymbol{w}^T \cdot \boldsymbol{x}'^{(i)}, y^{(i)}\right)$$

1. Define a loss function
2. Solve a potentially large and difficult optimization problem

→ The solution will reflect the properties of the loss function

# Loss functions for classification?

1. Define a **loss function**
2. Solve a potentially large and difficult optimization problem

→ The solution will reflect the properties of the loss function

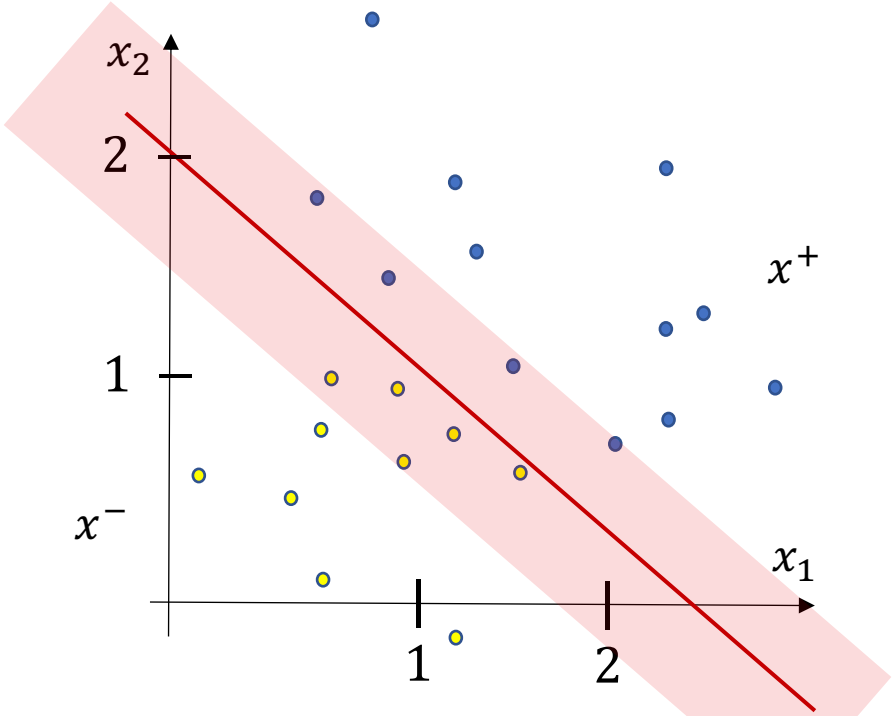
Let's first introduce a few useful notions: **Score** and **Margin**

# Where's the *critical region* for the classifier?

❖ *Given linearly separable data:*

✓ We expect that a classifier doesn't make mistakes for input data that lie **far away from the linear separator** (both sides)

➢ Classification errors might occur for inputs that lie ***about the linear separator*** *(both sides)*




$$f_{\boldsymbol{w}}(\boldsymbol{x}) = \frac{3}{4}x_1 + x_2 - 2 = 0$$

We can *quantify* this expected error rate / confidence of the classifier using the notion of **score**

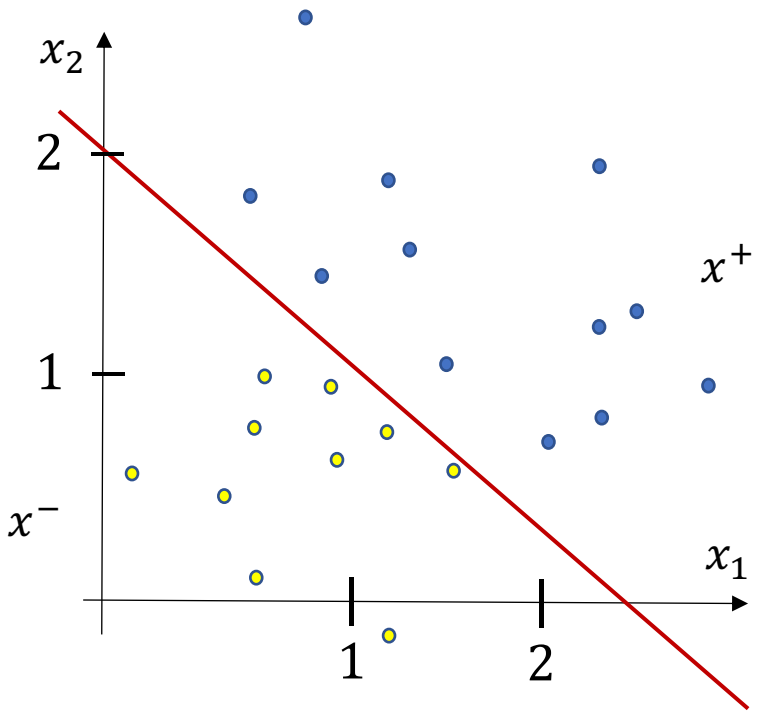
# Score: Classifier Behavior

- Given an example $(\boldsymbol{x}, y)$, the <u>value</u>

$$f_{\boldsymbol{w}}(\boldsymbol{x}) = \boldsymbol{w}^T \boldsymbol{x} + b$$

is the **score** of the classifier

- The score quantifies the *degree* to which the classification output is positive or negative
  - The absolute value of the score quantifies how close/far from the boundary the input point $\boldsymbol{x}$ is
  - The score represents how **confident** the classifier is making the prediction (based on its *line*)




$$f_{\boldsymbol{w}}(\boldsymbol{x}) = \frac{3}{4}x_1 + x_2 - 2 = 0$$

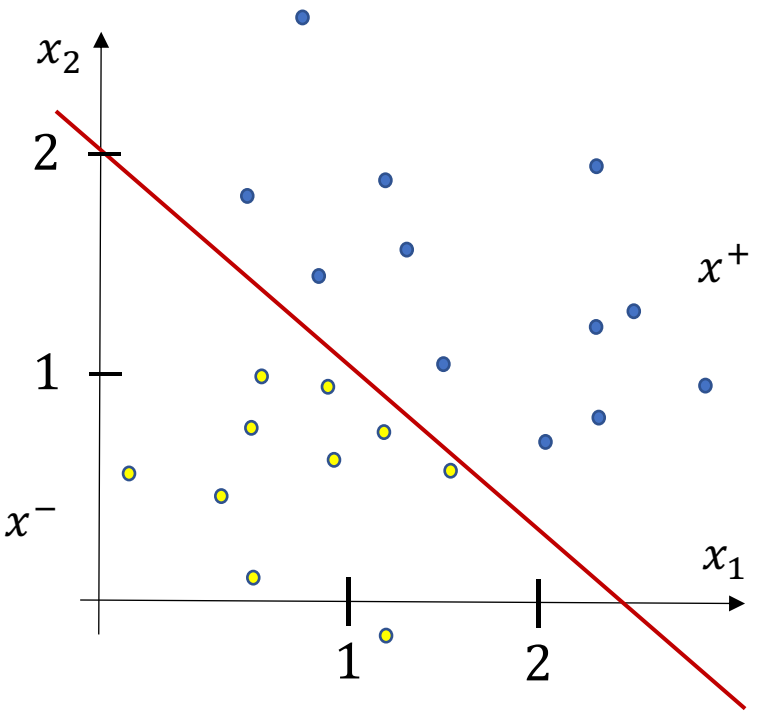
# Score: Classifier Behavior

- $f_{\boldsymbol{w}}((0,0)) = -2$
- $f_{\boldsymbol{w}}((-4,-2)) = -7$
- $f_{\boldsymbol{w}}((4,\ \ 2)) = 5$
- $f_{\boldsymbol{w}}((1,5)) = 15/4$

Far from boundary
→ High score

- $f_{\boldsymbol{w}}((1,1)) = -1/4$
- $f_{\boldsymbol{w}}((1.25,1)) = -\frac{1}{16}$
- $f_{\boldsymbol{w}}((1.5,1)) = 0.125$

Close to boundary
→ Low score




$$f_{\boldsymbol{w}}(\boldsymbol{x}) = \frac{3}{4}x_1 + x_2 - 2 = 0$$

# (Functional) Margin on an example

- The (functional) **margin** on an example $(\boldsymbol{x}, y)$, $y \in \{-1, +1\}$, is:

$$(\boldsymbol{w}^T \boldsymbol{x} + b)y$$

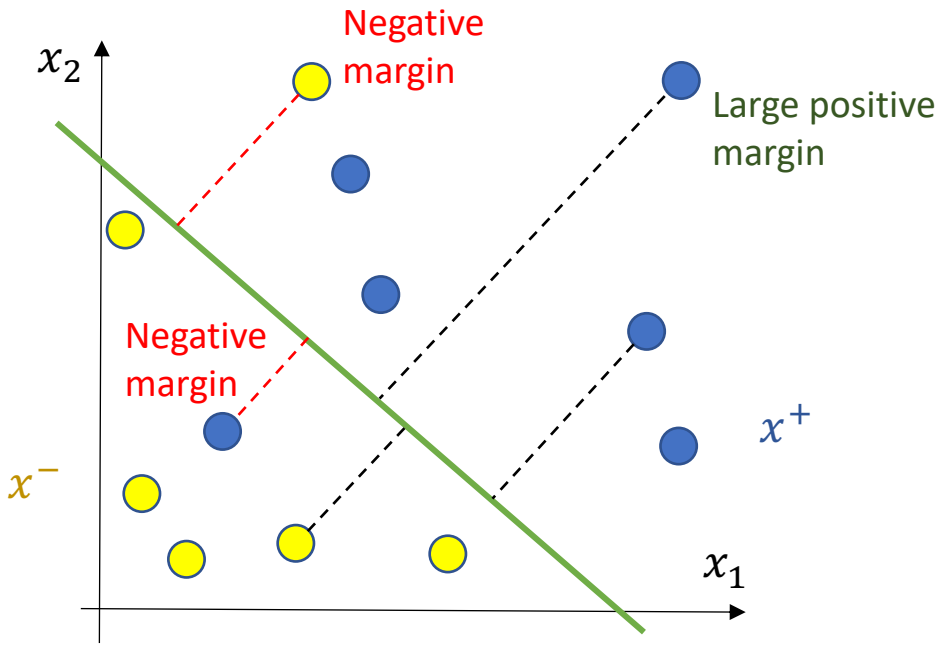
functional margin$(\boldsymbol{x})$ = score$(\boldsymbol{x})$ × true label$(\boldsymbol{x})$

- E.g., $y = -1$, $\boldsymbol{w}^T \boldsymbol{x} + b = 100$ → functional margin = $-100$
- → ***Classifier is very wrong***: sign of functional margin is negative, and its absolute value is quite large (e.g., decision boundary has on margin = 0), meaning that $\boldsymbol{x}$ is quite deep inside the $x^+$ region, while it had to be classified as $x^-$

# (Functional) Margin on an example

functional margin($\boldsymbol{x}$) = score($\boldsymbol{x}$) × true label($\boldsymbol{x}$)

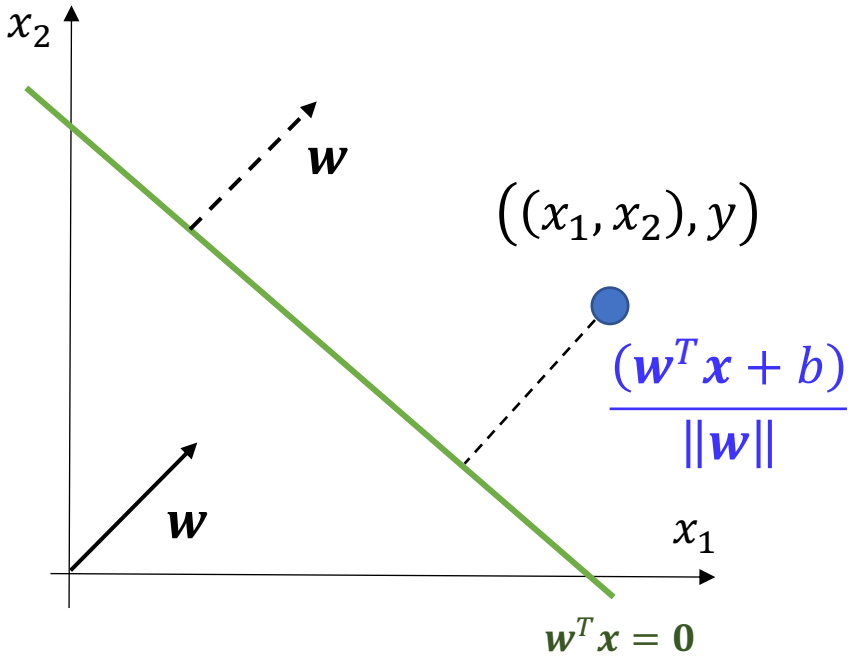
- Functional margin quantifies how (in)correct is the classification made by $f_{\boldsymbol{w}}(\boldsymbol{x})$
  - ✓ <u>The larger (positive) the margin, the better</u>
  - A <u>negative margin</u> corresponds to classification error



# Geometric margin of an example

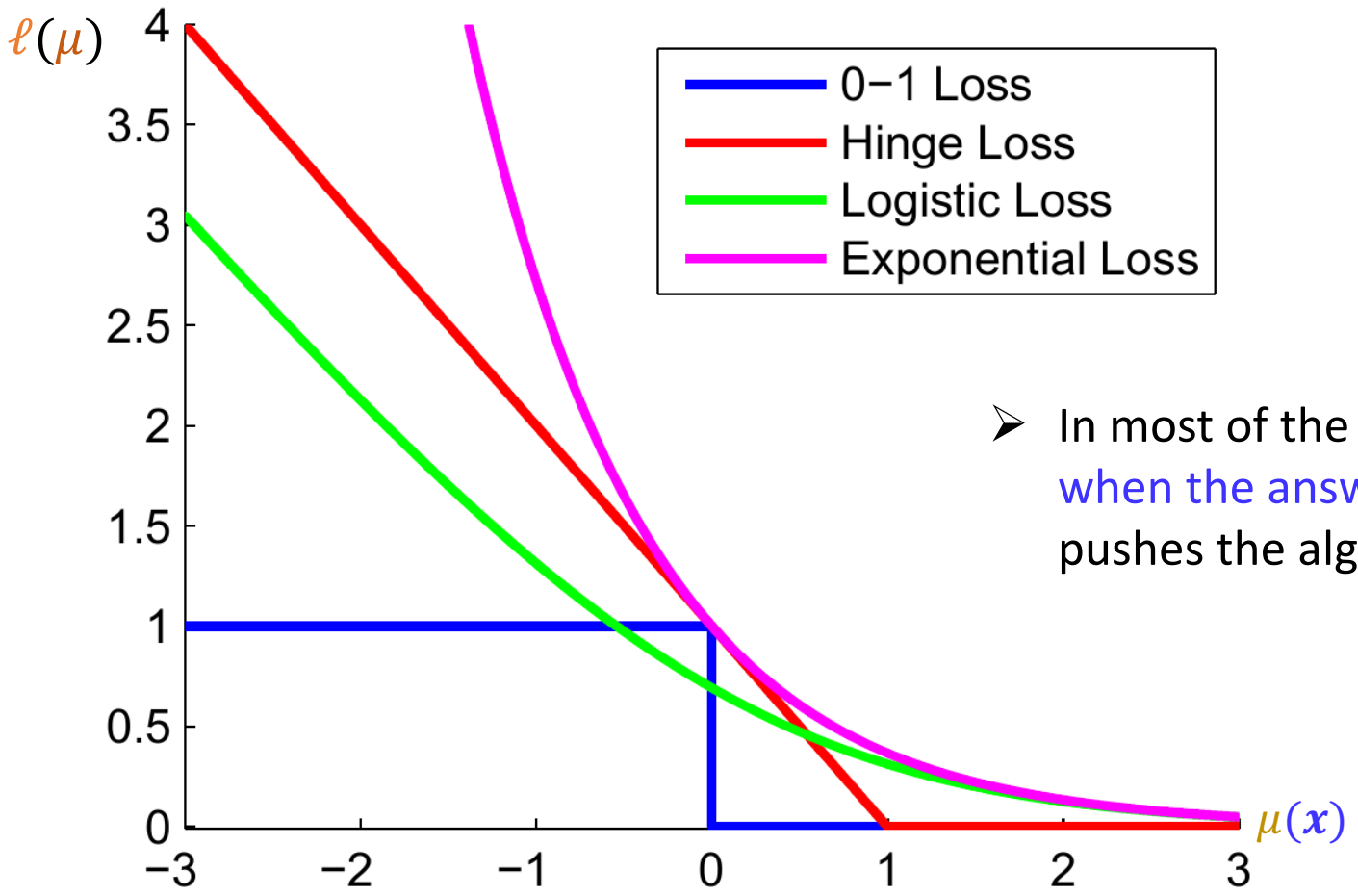
- **Geometric margin** of an example: (Euclidean) signed ($\pm 1$) distance of the example from the linear separator

$$\text{geometric margin}(\boldsymbol{x}) = \frac{\text{functional margin}(\boldsymbol{x})}{\textit{norm of } \boldsymbol{w}}$$




$$\frac{(\boldsymbol{w}^T\boldsymbol{x} + b)y}{\|\boldsymbol{w}\|}$$

# (Recap) Loss functions for classification




Losses (i.e., error corrections) of **different magnitude** in $(-\infty, +\infty)$ depending on <u>how bad (or good) the current prediction is!</u>

- In most of the cases, a loss is being generated even when the answer is basically correct, but the supervisor pushes the algorithm to learn an even better model!

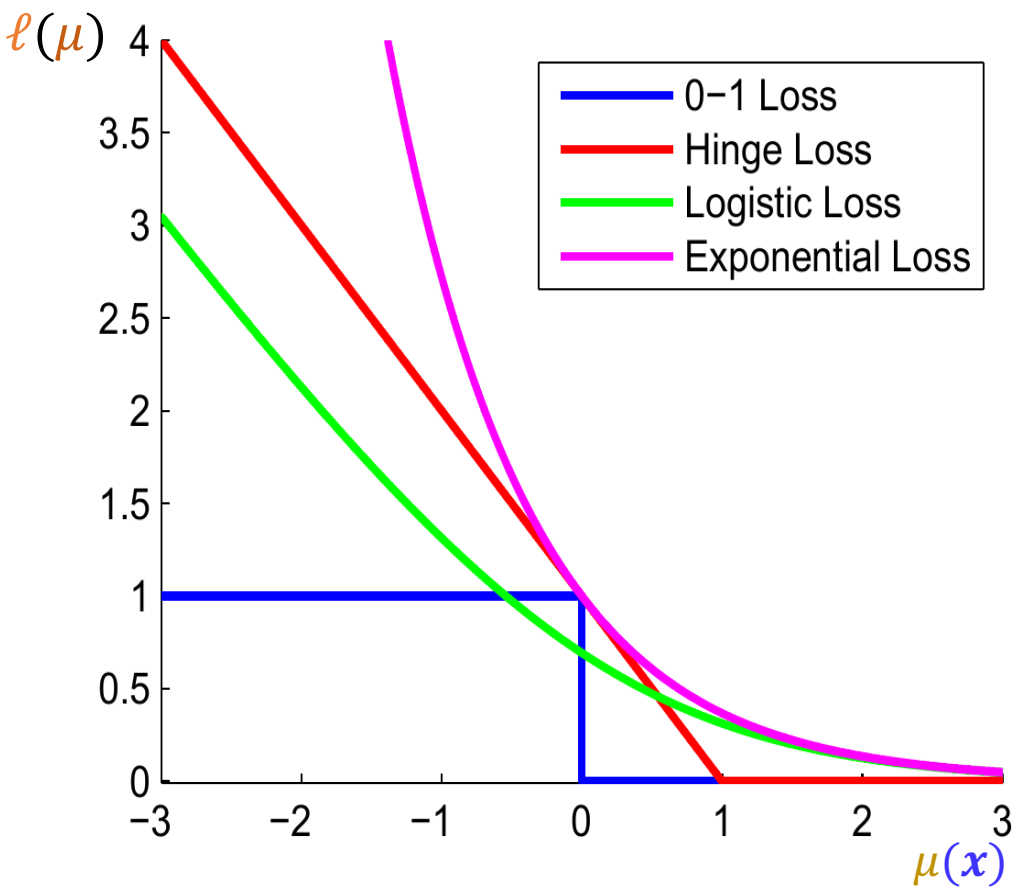
**Margin:** $\mu(\boldsymbol{x}) = f_{\boldsymbol{w}}(\boldsymbol{x}) \cdot y(\boldsymbol{x})$

✓ $\mu > 0 \rightarrow$ prediction agrees with label

○ $\mu < 0 \rightarrow$ prediction disagrees with label

**For linear models:** $\boxed{\mu(\boldsymbol{x}) = (\boldsymbol{w}^T\boldsymbol{x} + b)y(\boldsymbol{x})}$

# Loss functions for classification




- 0-1: $\ell(\mu(\boldsymbol{x})) = \begin{cases} 1 & \text{if } \mu(\boldsymbol{x}) < 0 \\ 0 & \text{otherwise} \end{cases}$
- Hinge: $\ell(\mu(\boldsymbol{x})) = \max\{1 - \mu(\boldsymbol{x}), 0\}$
- Logistic: $\ell(\mu(\boldsymbol{x})) = \log\left(1 + e^{-\mu(\boldsymbol{x})}\right)$
- Exponential: $\ell(\mu(\boldsymbol{x})) = e^{-\mu(\boldsymbol{x})}$

**For linear models:** $\boxed{\mu(\boldsymbol{x}) = (\boldsymbol{w}^T\boldsymbol{x} + b)y(\boldsymbol{x})}$

# How do we solve the optimization problem?

1. How do we use a linear classifier to **make predictions**?
   - Which **classes** correspond to the values $1, -\frac{5}{2}, 99, \ldots$ ?

2. **Given a loss function** $\ell$, how do we **train / learn** a linear classifier?
   - How do we find the ***best*** linear decision boundary?
   - The ***best***?

→ How do we solve the optimization problem?

$$\boldsymbol{w}^* = \min_{\boldsymbol{w}} \frac{1}{m} \sum_{i=1}^{m} \ell\left(\boldsymbol{w}^T \cdot \boldsymbol{x}'^{(i)}, y^{(i)}\right)$$

# How do we solve the optimization problem?

$$L_{emp}(\boldsymbol{w}) = \sum_{i=1}^{m} \ell\left(\boldsymbol{w}^T \cdot \boldsymbol{x}'^{(i)}, y^{(i)}\right)$$

$$\min_{\boldsymbol{w}} \; L_{emp}(\boldsymbol{w})$$

- How do we find the minimum of a function?
  1. Compute the 1st order **derivatives** and find where they are 0 (**stationary points**)
  2. Select the critical points that are of **minimum** using 2nd order derivatives and get the one with the **lowest value**

$L_{emp}$ is a multivariate function (d+1 variables!) → Compute the Gradient vector: $\nabla_{\boldsymbol{w}} L_{emp}(\boldsymbol{w}) = 0$

$$\begin{cases} \dfrac{\partial L_{emp}}{\partial w_0} = 0 \\ \dfrac{\partial L_{emp}}{\partial w_1} = 0 \\ \dfrac{\partial L_{emp}}{\partial w_2} = 0 \\ \vdots \\ \dfrac{\partial L_{emp}}{\partial w_d} = 0 \end{cases}$$

Solve the system of d+1 (possibly non-linear) equations →
… Check 2nd order partial derivatives (Hessian matrix) → Find $w^*$

# How do we solve the optimization problem?

$$L_{emp}(\boldsymbol{w}) = \sum_{i=1}^{m} \ell(\mu(\boldsymbol{x})) \qquad \mu(\boldsymbol{x}) = (\boldsymbol{w}^T \cdot \boldsymbol{x}')y$$

- 0-1: $\ell(\mu(\boldsymbol{x})) = \begin{cases} 1 & \text{if } \mu(\boldsymbol{x}) < 0 \\ 0 & \text{otherwise} \end{cases}$
- Hinge: $\ell(\mu(\boldsymbol{x})) = \max\{1 - \mu(\boldsymbol{x}), 0\}$
- Logistic: $\ell(\mu(\boldsymbol{x})) = \log\left(1 + e^{-\mu(\boldsymbol{x})}\right)$
- Exponential: $\ell(\mu(\boldsymbol{x})) = e^{-\mu(\boldsymbol{x})}$

$$\boxed{\min_{\boldsymbol{w}} L_{emp}(\boldsymbol{w})}$$

$$\begin{cases} \dfrac{\partial L_{emp}}{\partial w_0} = 0 \\ \dfrac{\partial L_{emp}}{\partial w_1} = 0 \\ \dfrac{\partial L_{emp}}{\partial w_2} = 0 \\ \vdots \\ \dfrac{\partial L_{emp}}{\partial w_d} = 0 \end{cases}$$

→ Compute the Gradient vector $\nabla_{\boldsymbol{w}} L_{emp}(\boldsymbol{w})$ ?

Solve the system of d+1 (possibly non-linear) equations →
… Check 2nd order partial derivatives (Hessian matrix) → Find $w^*$

**Can be very complex to solve analytically!** (even for linear models)

# How to compute gradients?

$$f(x_1, x_2) = x_1^2 + 3x_2^3 \qquad \frac{\partial f}{\partial x_1} = 2x_1 \qquad \frac{\partial f}{\partial x_2} = 9x_2^2$$

$$f(x_1, x_2, x_3) = x_1^2 x_2 x_3 + 3x_1 x_2 \qquad \frac{\partial f}{\partial x_1} = 2x_1 x_2 x_3 + 3x_2 \qquad \frac{\partial f}{\partial x_2} = x_1^2 x_3 + 3x_1 \qquad \frac{\partial f}{\partial x_3} = x_1^2 x_2$$

$$f(x_1, x_2, x_3, x_4) = 3\frac{\cos(x_1 x_4)\sin(x_2^5)}{e^{x_2} + (1 + x_2^2)/(x_1 x_2 x_4)} + 5x_1 x_3 x_4$$

$$\frac{\partial f}{\partial x_3}(a, b, c, d) \ ?$$

$$\frac{\partial f}{\partial x_3}(x_1, x_2, x_3, x_4) = 5x_1 x_4$$

$$\frac{\partial f}{\partial x_3}(a, b, c, d) = 5ad.$$

# How do we solve the optimization problem?

- Let's consider the *easy* case of using exponential loss $\ell(\mu(\boldsymbol{x})) = e^{-\mu(\boldsymbol{x})}$

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} \qquad \boldsymbol{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix} \qquad w_0 \equiv b$$

$$\mu(\boldsymbol{x}) = (\boldsymbol{w}^T\boldsymbol{x} + b)y(\boldsymbol{x})$$

$$L_{emp}(\boldsymbol{w}) = \sum_{i=1}^{m} e^{-(\boldsymbol{w}^T\boldsymbol{x}^i+b)y^i}$$

$$\nabla_{\boldsymbol{w}} L_{emp}(\boldsymbol{w}) = \begin{bmatrix} \frac{\partial L_{emp}}{\partial w_0} \\ \frac{\partial L_{emp}}{\partial w_1} \\ \frac{\partial L_{emp}}{\partial w_2} \\ \vdots \\ \frac{\partial L_{emp}}{\partial w_d} \end{bmatrix}$$

$$\frac{\partial L_{emp}}{\partial w_0} = \sum_{i=1}^{m} -y^i e^{-(\boldsymbol{w}^T\boldsymbol{x}^i+b)y^i}$$

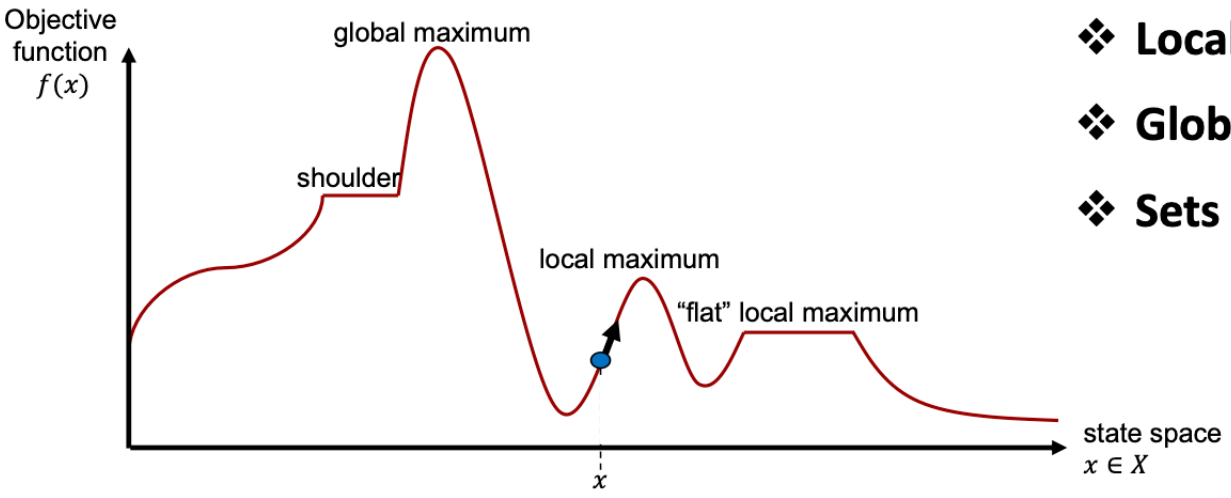
$$\frac{\partial L_{emp}}{\partial w_k} = \sum_{i=1}^{m} -y^i x_k^i e^{-(\boldsymbol{w}^T\boldsymbol{x}^i+b)y^i}$$

$$\nabla_{\boldsymbol{w}} L_{emp}(\boldsymbol{w}) = 0 \quad \begin{cases} \sum_{i=1}^{m} -y^i e^{-(\boldsymbol{w}^T\boldsymbol{x}^i+b)y^i} = 0 \\ \sum_{i=1}^{m} -y^i x_1^i e^{-(\boldsymbol{w}^T\boldsymbol{x}^i+b)y^i} = 0 \\ \sum_{i=1}^{m} -y^i x_2^i e^{-(\boldsymbol{w}^T\boldsymbol{x}^i+b)y^i} = 0 \\ \vdots \\ \sum_{i=1}^{m} -y^i x_d^i e^{-(\boldsymbol{w}^T\boldsymbol{x}^i+b)y^i} = 0 \end{cases}$$

# How do we solve the optimization problem?

**Can be very complex to solve analytically!** (even for linear models)

Other approach: **numerically** use an iterative approach to find the stationary points




- **Local** minima / maxima
- **Global** minima / maxima
- **Sets** of points of local/global minima/maxima

To be continued …